

REDSPHER

Full Stack Developer

Technikai interjúfelkészítő

Technikai körre optimalizálva

TypeScript - React - Node.js - REST - AWS -
CI/CD - observability - system design - LLM -
MCP - AI agents

Telefonon jól olvasható, egyoszlopos kiadás
Magyar magyarázatok, angol szakkifejezésekkel
2026. július

Contents

1	0. Mire számíts a technikai körben	3
1.1	Valószínű interjústruktúra	3
1.2	Válaszformula technikai kérdéshez . .	4
2	1. TypeScript - várható mélyítés	5
3	2. React - rendering, state és production UI	13
4	3. Node.js - event loop, hibakezelés és skálázás	22
5	4. REST, HTTP, adat és security	30
6	5. AWS és cloud architecture	37
7	6. CI/CD, tesztelés és production hibakeresés	43
8	7. LLM, MCP és AI agents - gyakorlati mélység	46
9	8. System design - hogyan vezesd a beszélgetést	52
9.1	System design válaszváz	52
9.2	Gyors kapacitásbecslés	55
109.	Saját technikai történetbank	56
1110.	Live coding és code review gyakorlás	59

11.1	Komplexitási gyorslista	62
12	11. 60 perces próbainterjú	63
12.1	Interjúztató follow-up csomag	63
13	12. Utolsó napi technikai cheat sheet	64
13.1	Öt saját példa, amelyet biztosan gyakorolj	65
13.2	Kérdések, amelyeket te tehetsz fel . .	65
14	13. Források és hatókör	66

0. Mire számíts a technikai körben

Ez a kiadás nem recruiter- vagy általános behavior interjúra készült. A hangsúly a technikai mélységen, a döntések indoklásán, a kódrészleteken, a production gondolkodáson és a system designon van. A hirdetés alapján a legerősebb fókusz: React + TypeScript, Node.js backend, REST, AWS, CI/CD, architektúrális döntések, valamint AI agents és MCP.

Valószínű interjústruktúra

- 5-10 perc: technikai bemutatkozás és egy releváns projekt mély bemutatása.
- 20-30 perc: React, TypeScript és Node.js kérdések, follow-upokkal.
- 10-20 perc: API, AWS, deployment, hibakezelés és production problémák.
- 15-30 perc: system design vagy közös architektúra-gondolkodás.
- Opcionálisan: rövid live coding, code review vagy pszeudokódos feladat.
- AI/MCP rész: inkább gyakorlati integrációs gondolkodás, biztonság és orchestration, nem modellkutatás.

Válaszformula technikai kérdéshez

- 1. Rövid definíció** - egy-két mondat.
- 2. Mikor használnád** - konkrét helyzet.
- 3. Trade-off** - mit nyersz és mi az ára.
- 4. Production részlet** - hiba, monitoring, skálázás vagy security.
- 5. Saját példa** - ahol valóban csináltál hasonlót.

ESSENTIAL Hogyan mutatnád be magad technikai interjún 60 másodpercben?

I am a full-stack developer with more than fifteen years of experience, with my strongest recent focus on React, TypeScript and Node.js. I have built user-facing applications and scalable real-time backend services, and I have worked across architecture, implementation, releases, monitoring and production troubleshooting. A recurring theme in my work is simplifying systems and automating repetitive processes. I have also been deepening my AWS knowledge and experimenting with practical LLM and agent-based workflows, so the combination of end-to-end product work, cloud and AI integration in this role is a strong match. **Várható follow-upok**

- Which project best represents your full-stack experience?
- What was the hardest production problem you solved?
- Where are you strongest: frontend, backend or architecture?

Buktató Ne sorolj fel húsz technológiát. Az ownership, a scale, a production felelősség és a problémamegoldás legyen a történet.

ESSENTIAL Melyik projektet választanád első technikai deep dive-nak?

A skálázható WebSocket support chat jó első történet: legacy protokoll reverse engineering, JWT authentication, Redis pub/sub, horizontal scaling, connection lifecycle, latency és production rollout. Ez egyszerre mutat Node.js, distributed systems, security, observability és pragmatikus migrációs tapasztalatot. React fókusz esetén második történetként a nagy TypeScript SPA-t és az end-to-end release ownershipot hozd. **Várható follow-upok**

- Mi volt a bottleneck?
- Hogyan kezelted a reconnectet és duplicate eventeket?
- Hogyan tesztelted terhelés alatt?
- Mit csinálnál ma másképp?

Buktató Ne csak az architektúrát mondd el. Legyen konkrét döntés, alternatíva, mérés és probléma is.

1. TypeScript - várható mélyítés

A strong TypeScript elvárás miatt nem elég a szintaxis. Az interjúztató azt keresi, hogy

biztonságos API-határokat, jól modellezett state-et és karbantartható absztrakciókat tudsz-e kialakítani.

ESSENTIAL Mi a TypeScript valódi értéke egy nagy rendszerben?

A TypeScript fejlesztési időben ellenőrizhető szerződéseket ad modulok, komponensek és API-határok között. A legnagyobb előnye a biztonságosabb refaktorálás, jobb IDE-visszajelzés és pontosabb domain modellezés. Nem garantál runtime helyességet: a hálózatról, adatbázisból vagy user inputból érkező adatot külön validálni kell. **Várható follow-upok**

- Mikor nem éri meg túlkomplicálni a típusokat?
- Hogyan osztod meg a típusokat frontend és backend között?
- Mitől lesz egy type definition túl clever?

Buktató Ne mondd, hogy a TypeScript megszünteti a runtime hibákat.

ESSENTIAL Mi a különbség az any, unknown és never között?

Az any kikapcsolja a type checkinget az adott értéken. Az unknown ismeretlen értéket jelent, amelyet használat előtt narrowolni vagy validálni kell. A never olyan állapotot jelöl, amely nem fordulhat elő, például egy exhaustively kezelt discriminated union maradék ága vagy egy mindig hibát dobó függvény visszatérése.

Várható follow-upok

- Mi legyen a catch változó típusa?
- Mutass exhaustive switch példát.
- Mikor indokolt az any?

Buktató Az unknown után ne type assertionnel ugord át automatikusan az ellenőrzést.

ESSENTIAL Hogyan modelleznél többállapotú aszinkron UI-t?

Discriminated unionnal, például idle, loading, success és error állapotokkal. Így a state kombinációk közül csak az érvényesek reprezentálhatók, és switch esetén exhaustive ellenőrzés is készíthető. Ez jobb, mint több független boolean, amelyekből lehet loading=true és error=true egyszerre.

Várható follow-upok

- Mi kerülne a success és error ágba?
- Hogyan működik a narrowing?
- Reducerrel vagy több useState-tel kezelned?

Buktató Kerüld az egymástól független isLoading, hasError, data flag-kombinációkat.

ESSENTIAL Mikor használsz generics-et, és mikor uniont?

Generic akkor jó, ha a bemenet és kimenet típusai között kapcsolatot kell megőrizni, például identity, repository vagy typed utility esetén. Union akkor jobb, ha véges számú konkrét variációt modellezünk. Ha a generic

nem köt össze semmit, csak elrejtí az any-t, valószínűleg rossz absztrakció. **Várható follow-upok**

- Mit jelent a T extends constraint?
- Mutass keyof alapú property accessort.
- Mikor használnál overloadot?

Buktató Ne vezess be generic type paramétert csak azért, hogy a függvény „advancednek” tűnjön.

VALÓSZÍNŰ Mi a structural typing és milyen meglepetést okozhat?

TypeScriptben a kompatibilitást főleg a shape határozza meg, nem a deklarált név. Emiatt két külön interface kompatibilis lehet, ha megfelelő mezők vannak. Object literal esetén excess property check segít, de köztes változón keresztül lazább lehet az ellenőrzés. Domain szempontból azonos primitívek, például UserId és ShipmentId megkülönböztetéséhez branded type használható. **Várható follow-upok**

- Miért más az object literal és egy változó ellenőrzése?
- Mikor használnál branded type-ot?

Buktató Ne várj Java- vagy C#-szerű nominal typingot alapértelmezetten.

VALÓSZÍNŰ Mi a különbség type és interface között?

Mindkettő képes object shape leírására. Interface jól bővíthető és declaration merginget támogat. Type alias rugalmasabb

union, intersection, conditional és mapped type esetén. Csapatkonvenció fontosabb, mint egy univerzális szabály; public object API-nál interface, összetett type compositionnál type gyakori választás.

Várható follow-upok

- Mi az a declaration merging?
- Lehet-e uniont interface-szel modellezni?

Buktató Ne állítsd, hogy az egyik minden helyzetben jobb.

VALÓSZÍNŰ Mire valók a mapped és conditional type-ok?

Mapped type meglévő kulcshalmazból állít elő új object type-ot, például readonly vagy optional változatot. Conditional type feltételes transzformációt ad a type systemben, gyakran inferrel kombinálva. Hasznosak library vagy shared infrastructure kódban, de üzleti kódban a túl mély type-level programozás rontja az olvashatóságot és a compiler teljesítményét.

Várható follow-upok

- Hogyan működik a distributive conditional type?
- Mit csinál az infer?
- Mikor írnál saját utility type-ot?

Buktató A típus ne legyen nehezebben érthető, mint a runtime logika.

VALÓSZÍNŰ Mit jelent a variance callbackeknél?

A variance azt írja le, hogyan öröklődik a kompatibilitás generic típusokon keresztül. Callback paramétereknél a contravariance fontos: egy szűkebb eseményt kezelő függvény nem feltétlenül biztonságos ott, ahol tágabb esemény jöhet. `strictFunctionTypes` segít ilyen hibák felismerésében. Interjún a biztonsági intuíció fontosabb, mint a formális jelölés. **Várható follow-upok**

- Miért lehet veszélyes egy túl szűk callback parameter?
- Mi a covariance egy readonly listánál?

Buktató Ne vessz el típuselméleti definíciókban; mutass konkrét callback példát.

ESSENTIAL Hogyan validálnád a backendből érkező adatot?

A response kezdetben unknown. A boundary rétegben schema validatorral vagy kézzel ellenőrzöm a shape-et, majd csak validált domain type kerül beljebb. A validation error legyen megfigyelhető, de ne logoljunk érzékeny adatot. A frontend és backend közös TypeScript típusa önmagában nem védi a runtime szerződést. **Várható follow-upok**

- Zod, JSON Schema vagy OpenAPI generated client?
- Mi történjen részben hibás listánál?
- Hogyan verziózd a schemát?

Buktató Az `as Shipment[]` nem runtime validáció.

VALÓSZÍNŰ Hogyan konfigurálnád a `tsconfig`-ot szigorú projekthez?

`strict` legyen bekapcsolva. Érdemes megfontolni a `noUncheckedIndexedAccess`, `exactOptionalPropertyTypes` és `useUnknownInCatchVariables` opciókat. ESM/CJS és `moduleResolution` beállításnak a futtatókörnyezettel és bundlerrel összhangban kell lennie. A cél nem minden `flag`-ot bekapcsolása, hanem konzisztens `build` és fokozatosan fenntartható `strictness`.

Várható follow-upok

- Mit változtat a `strictNullChecks`?
- Mit okozhat az `exactOptionalPropertyTypes`?
- Hogyan migrálnál legacy projektet `strict` módra?

Buktató Ne válaszolj kizárólag opciólistával; mondd el a migrációs stratégiát is.

VALÓSZÍNŰ Mi a különbség ESM és CommonJS között Node.js-ban?

ESM statikus `import/export` modellt használ, támogatja a `top-level await`-et és a modern tooling alapja. CommonJS `require/module.exports` alapú és történelmileg a Node alapértelmezettje volt. A problémák többnyire interoperábilisból, `package type`-ből, `file extension`-ből, `default import`-ból és `test/build tooling`-ből jönnek. Egy projekten belül egyértelmű modultípust

választanék. **Várható follow-upok**

- Mi az a package.json type mező?
- Miért nehéz egyes CJS csomagok ESM importja?

Buktató Ne keverd a TypeScript module settinget a runtime tényleges module systemével.

KÓDOLÁSI FELADAT Exhaustive reducer

Feladat Írj typed reducert shipment állapothoz: started, succeeded, failed. Ismeretlen action ne maradjon észrevétlen. **Mit figyelnek**

- Discriminated union
- Exhaustive never check
- Immutable update
- Nincs any

```
type Action =
  | { type: "started" }
  | {
    type: "succeeded";
    data: Shipment[];
  }
  | {
    type: "failed";
    message: string;
  };

function assertNever(x: never): never {
  const detail = JSON.stringify(x);
  throw new Error(`Unhandled: ${detail}`);
}

function reducer(
  state: State,
  action: Action,
): State {
  switch (action.type) {
    case "started":
      return { status: "loading" };
  }
```

```

    case "succeeded":
      return {
        status: "success",
        data: action.data,
      };
    case "failed":
      return {
        status: "error",
        message: action.message,
      };
    default:
      return assertNever(action);
  }
}

```

Megbeszélendő trade-offok

- Runtimeon a default ág továbbra is védhet külső inputtól.
- A reducer típusbiztonsága nem helyettesíti az API action validációját.

2. React - rendering, state és production UI

A technikai körben valószínűleg nem hook-definíciókat kérnek, hanem azt, hogy érted-e a renderelést, effect lifecycle-t, state modellezést, performance-t és a hibás UI állapotok megelőzését.

ESSENTIAL Mi vált ki renderelést Reactben?

A komponens renderel, amikor a saját state-je frissül, a parent renderel és új propsot ad, vagy egy olvasott context/external store érték változik. A render fázisnak tisztának kell lennie: ugyanabból az inputból ugyanazt a

UI-leírást adja. A DOM frissítése a commit fázisban történik. A render nem jelenti automatikusan azt, hogy minden DOM node újra létrejön. **Várható follow-upok**

- Mi a render és commit fázis?
- Mikor renderel újra egy memo komponens?
- Miért baj a side effect render közben?

Buktató Ne mondd, hogy minden render teljes DOM újraépítés.

ESSENTIAL Miért fontos a key és miért veszélyes az index?

A key ad stabil identitást a sibling elemeknek, hogy React meg tudja őrizni vagy helyesen újra létrehozni a state-et. Index key akkor problémás, ha a lista rendeződik, beszúrunk vagy törölünk: egy komponens state-je másik adatelemhez tapadhat. Stabil domain ID a jó alapértelmezés. **Várható follow-upok**

- Mikor elfogadható az index?
- Mit okoz a Math.random key?
- Miért nem kapja meg a komponens propként a key-t?

Buktató Ne csak performance-problémaként írd le; state correctness probléma is.

ESSENTIAL Hogyan gondolkodsz a state-ről mint snapshotról?

Egy render eseménykezelői az adott render state snapshotját látják. A setter

a következő renderre kér frissítést, nem módosítja azonnal a lokális változót. Több egymásra épülő frissítésnél functional updater kell. Ez magyarázza a stale state és batching jelenségeket. **Várható follow-upok**

- Miért ad három `setCount(count + 1)` csak egyet?
- Mikor kell `setCount(c => c + 1)`?

Buktató Ne kezeld a state settert szinkron assignmentként.

ESSENTIAL Mikor kell `useEffect` és mikor nem?

Effect külső rendszerrel történő szinkronizációra való: network subscription, timer, DOM API, third-party widget. Derived value kiszámítására vagy event kezelésére általában nem kell. Ha egy érték propsból és state-ből render közben kiszámítható, ne tárold új state-ben és ne effecttel szinkronizáld. **Várható follow-upok**

- Mi számít external systemnek?
- Hogyan törölsz subscriptiont?
- Miért fut development Strict Mode-ban extra setup/cleanup?

Buktató Ne használd az effectet általános „amikor valami változik” mechanizmusként.

ESSENTIAL Mitől hibás egy effect dependency array?

Az effectben olvasott reactive értékeknek dependencyként szerepelniük kell. Hiány

esetén stale closure alakulhat ki. A javítás nem a lint kikapcsolása, hanem a logika átszervezése: eventbe mozgatás, updater function, stabil referencia, effect szétválasztása vagy szükségtelen effect eltávolítása. **Várható follow-upok**

- Mi a stale closure?
- Mikor lehet object dependency problémás?
- Mit csinál az effect cleanup?

Buktató A dependency array nem kézi optimalizációs lista.

ESSENTIAL Hogyan kezeled a request race conditiont Reactben?

Minden query változáskor indított requestet AbortControllerrel leállítok, vagy request ID/generation alapján csak a legfrissebb eredményt fogadom el. Cleanupban abortálok. Emellett a loading, error és empty state-et explicit modellezem. Data fetching library esetén a cache és deduplication szabályait ismerni kell.

Várható follow-upok

- Mi történik gyors gépelésnél?
- Abort mindig garantálja, hogy nincs response handler?
- Hogyan kezelnéd a retryt?

Buktató Ne engedd, hogy egy régebbi lassú response felülírja az újabbat.

VALÓSZÍNŰ Mikor használsd a useMemo, useCallback és React.memo eszközöket?

A memoization konkrét mérés alapján indokolt: drága számítás, stabil prop egy memo childnak vagy dependency stabilizálása. Nem correctness eszköz és nem ingyenes; összehasonlítási és memória költsége van. Előbb a state localityt, component boundaryt és unnecessary effecteket javítom, majd profilerrel mérek.

Várható follow-upok

- Mikor nem segít a useCallback?
- Mi töri meg a React.memo-t?
- Hogyan használnád a Profilert?

Buktató Ne memoizálj automatikusan minden függvényt és objectet.

VALÓSZÍNŰ Context vagy external state store?

Context jó ritkán változó, széles körben szükséges értékekhez, például theme vagy auth session. Nagy frekvenciájú, részleteiben fogyasztott globális state-nél selectoros external store hatékonyabb lehet. Először state locality és composition; csak a valóban megosztott state kerüljön globális helyre. **Várható follow-upok**

- Miért renderelhet sok fogyasztó context változáskor?
- Mi a server state és client state különbsége?

Buktató Ne tegyél minden state-et globális store-ba.

VALÓSZÍNŰ Controlled vagy uncontrolled form?

Controlled inputnál React state a source of truth, ami jó dinamikus validationhoz és összefüggő UI-hoz. Uncontrolled esetén a DOM tárolja az értéket, ref vagy form submit olvassa ki; egyszerű vagy nagy formoknál kevesebb render lehet. Sok form library hibrid megközelítést használ. **Várható follow-upok**

- Hogyan kezelnél 100 mezős formot?
- Mikor kell debounced validation?
- Hogyan mutatnád a server-side hibát?

Buktató Ne állítsd, hogy minden inputnak controllednek kell lennie.

VALÓSZÍNŰ Hogyan tervezel reusable komponenst?

Stabil, szűk API-t adok, compositiont preferálok túl sok boolean prop helyett, és elkülönítem a vizuális primitive-et a domain komponensektől. Kezelem az accessibilityt, loading/disabled/error állapotokat, ref forwardingot csak szükség esetén, és Storybookkal dokumentálom a variációkat.

Várható follow-upok

- Mi a prop explosion?
- Mikor jó a compound component pattern?
- Hogyan verziózol design system komponenst?

Saját irányod A Storybook, Figma együttműködés és reusable component tapasztalat jó konkrét példa. **Buktató** Ne

absztrahálj egyetlen használat után anélkül, hogy látnád a valódi variációkat.

VALÓSZÍNŰ Hogyan kezelnél hibát React alkalmazásban?

Render hibára error boundary kell megfelelő fallbackkel és logginggal. Event handler vagy async request hibát explicit try/catch és state kezeli. A usernek érthető, helyreállítható UI-t mutatok; a logba correlation ID, route, release és releváns context kerül, személyes adat nélkül.

Várható follow-upok

- Mit nem kap el az error boundary?
- Hol helyeznél el boundaryket?
- Hogyan kötnéd Sentryhez?

Saját irányod A Sentry és Kibana production tapasztalatot érdemes ide kötni. **Buktató** Ne legyen egyetlen globális boundary, amely minden hibánál teljesen fehér képernyőt ad.

DEEP DIVE Mit jelent a concurrent rendering szemlélete?

React képes a render munkát megszakítani, újratekinteni és prioritás szerint ütemezni. Emiatt a rendernek tisztának és idempotensnek kell lennie. Transitionnal egy nem sürgős UI frissítés alacsonyabb prioritású lehet; Suspense boundary kontrollált fallbacket ad. Ez nem automatikus multithreading és nem gyorsít minden alkalmazást. **Várható follow-upok**

- Mi a sürgős és transition update?
- Miért fontos a pure render?
- Mikor segít a Suspense?

Buktató Ne ígérj automatikus performance javulást concurrent feature-öktől.

VALÓSZÍNŰ Hogyan tesztelnél React komponenst?

A user által látható viselkedést tesztelem: renderelt szöveg, role, interakció és hálózati hatás. Kerülöm az implementation detailt és a state közvetlen vizsgálatát. Unit teszt a tiszta logikának, component/integration teszt a UI-flow-nak, kevés E2E a kritikus üzleti útvonalaknak. **Várható follow-upok**

- Mit mockolnál?
- Hogyan tesztelnél loading és error state-et?
- Miért jobb a role alapú query?

Buktató Ne legyen snapshot az egyetlen ellenőrzés.

KÓDOLÁSI FELADAT Kereshető lista cancellationnel

Feladat Készíts komponenst, amely gépelés után lekéri a shipment listát. Ne villogjon régi eredmény, legyen loading és error state.

Mit figyelnek

- Debounce vagy deferred input indoklása
- AbortController cleanup
- Race condition elkerülése

- Accessible loading/error UI
- Stabil key

```
function ShipmentSearch() {
  const [query, setQuery] =
    useState("");
  const [state, setState] =
    useState({ status: "idle" });

  useEffect(() => {
    if (!query.trim()) {
      setState({ status: "idle" });
      return;
    }

    const controller =
      new AbortController();
    const encoded =
      encodeURIComponent(query);
    const url =
      `/api/shipments?q=${encoded}`;

    setState({ status: "loading" });

    fetch(url, {
      signal: controller.signal,
    })
      .then(response => {
        if (!response.ok) {
          throw new Error("HTTP error");
        }
        return response.json();
      })
      .then(data => setState({
        status: "success",
        data,
      }))
      .catch(error => {
        if (error.name !== "AbortError") {
          setState({
            status: "error",
            error,
          });
        }
      });

    return () => controller.abort();
  }, [query]);
}
```

Megbeszélendő trade-offok

- Productionben schema validation és cache library is indokolt lehet.

- Debounce külön effecttel vagy custom hookkal tehető hozzá.

3. Node.js - event loop, hibakezelés és skálázás

A saját háttérrel miatt itt mély follow-upokra számíthatsz. Az interjúztató valószínűleg production példát kér: mitől blokkol, hogyan skálázol, hogyan áll le biztonságosan, és mit mérsz.

ESSENTIAL Hogyan működik magas szinten a Node.js event loop?

A JavaScript callbackek egy fő event loop szálon futnak. A nem blokkoló I/O-t az operációs rendszer és libuv kezeli; bizonyos műveletek a libuv worker poolba kerülnek. Amikor egy művelet elkészül, callback vagy promise continuation kerül feldolgozásra. A skálázhatóság feltétele, hogy egy callback rövid ideig foglalja a fő szálát. **Várható follow-upok**

- Mi kerül worker poolba?
- Mi a microtask queue?
- Mikor fut a process.nextTick?
- Mi történik CPU-intenzív JSON feldolgozásnál?

Buktató Ne mondd, hogy Node.js egyszerűen „single-threaded”; a JavaScript fő szál és a háttér I/O/worker mechanizmus külön fogalom.

ESSENTIAL Mi blokkolhatja az event loopot?

Szinkron file API, nagy CPU-loop, drága regex, óriási JSON parse/stringify, túl nagy synchronous serialization vagy rosszul méretezett callback. Ennek hatása minden kapcsolat latencyjére kiterjed. Mérhető event loop laggal, p95/p99 latencyvel, CPU profile-lal és flame graphgal. **Várható follow-upok**

- Hogyan javítanád a CPU-intenzív feladatot?
- Mit okoz egy ReDoS regex?
- Mikor streamelnél?

Buktató Az await önmagában nem tesz CPU-munkát nem blokkolóvá.

ESSENTIAL Worker threads, cluster vagy több container?

Worker thread CPU-intenzív JavaScript párhuzamosítására jó, lehetőleg poolban. Cluster vagy több Node process több CPU mag kihasználását adja, de memóriát nem oszt meg egyszerűen. Modern productionben gyakran több stateless container vagy process fut load balancer mögött; shared state külső store-ba kerül. I/O-intenzív feladathoz nem worker thread az első megoldás. **Várható follow-upok**

- Hogyan kezelnéd a sticky sessiont WebSocketnél?
- Mit oszthat meg worker thread?
- Mi a spawn overhead?

Saját irányod A PM2 clustering és horizontálisan skálázott WebSocket rendszered erős példa. **Buktató** Ne használd a worker threadet minden async feladatra.

ESSENTIAL Hogyan kezeled a hibákat egy Node API-ban?

Elkülönítem az expected operational errort és a programmer bugot. Domain/service réteg typed vagy klasszifikált hibát ad, a HTTP boundary egységes error response-ra fordítja, logolja correlation ID-val, és nem szivárogtat stack trace-t a kliensnek. Ismeretlen állapotú processnél kontrollált leállítás és restart biztonságosabb lehet, mint a vak folytatás. **Várható follow-upok**

- Mit csinálsz unhandledRejectionnél?
- Mikor retryolható egy hiba?
- Mi legyen a 4xx és 5xx különbség?

Buktató Ne nyelj le hibát üres catch blokkal, és ne adj mindenre 500-at kontextus nélkül.

ESSENTIAL Mi a graceful shutdown menete?

SIGTERM esetén leállítom az új forgalom fogadását, readiness false lesz, befejezem vagy timeouttal lezárom az aktív kéréseket, leállítom a queue consumeröket, flusholom a log/trace buffert és zárom a DB connection poolt. Van hard timeout, mert végtelen drain sem elfogadható. **Várható follow-upok**

- Mi történik Kubernetes rolling update alatt?

- Hogyan zárod a WebSocketeket?
- Mi lesz a folyamatban lévő jobokkal?

Buktató Ne csak `process.exit(0)`-t hívj azonnal.

ESSENTIAL Mi a stream és mi a backpressure?

Streammel az adatot darabokban dolgozzuk fel, nem kell mindent memóriába tölteni. Backpressure akkor kell, amikor a producer gyorsabb a consumernél; a writable jelzi, hogy várni kell, a pipeline pedig továbbítja a hibát és kezeli a lezárást. Nagy file, export, proxy vagy transform esetén ez fontos.

Várható follow-upok

- Miért veszélyes a data event kontroll nélkül?
- Mit csinál a `highWaterMark`?
- Miért jó a pipeline?

Buktató Ne feltételezd, hogy stream automatikusan megold minden memória- és sebességproblémát.

VALÓSZÍNŰ Hogyan limitáld az aszinkron konkurenciát?

Nem indítok korlátlan `Promise.all`-t nagy listára. Worker pool vagy semaphore jellegű concurrency limitet használok, figyelembe véve a downstream rate limitet, DB connection poolt és memóriát. A retry exponential backoff plus jitterrel történik, csak idempotens vagy deduplikált műveletnél. **Várható follow-upok**

- Miért veszélyes a Promise.all 100 000 elemre?
- Mi a Promise.allSettled szerepe?
- Hogyan kezelsz partial failure-t?

Buktató A concurrency nem azonos a parallelismussal.

VALÓSZÍNŰ Hogyan építesz rétegezett Node backendet?

A transport/controller a HTTP részleteket kezeli, application/service réteg az use case-t, domain réteg az üzleti invariantokat, repository/integration adapter pedig az I/O-t. A dependency irány a domain felé mutat. Nem kell dogmatikus clean architecture: a cél tesztelhető határok és kevés framework coupling. **Várható follow-upok**

- Hol validálsz?
- Hol kezdődik a transaction?
- Mikor túlzás a repository pattern?

Buktató Ne hozz létre üres pass-through rétegeket valódi felelősség nélkül.

ESSENTIAL Hogyan skáláznál WebSocket szolgáltatást több instance-ra?

A connection egy konkrét instance-hoz tartozik, de a routing és fan-out megosztott infrastruktúrát igényel. Load balancer mögött több stateless gateway fut; user-to-connection registry vagy partitioning segít a célzásban, pub/sub vagy broker

terjeszti az eventeket. Reconnect, heartbeat, backpressure, ordering, duplicate és slow consumer kezelés szükséges. **Várható follow-upok**

- Kell-e sticky session?
- Mi történik instance kiesésnél?
- Hogyan garantálod az orderinget?
- Miért lehet Redis pub/sub kevés tartós deliveryhez?

Saját irányod A Redis pub/sub alapú support chat és a nagy concurrency broadcaster közvetlenül ide illik. **Buktató** Ne állíts exactly-once deliveryt egyszerű pub/sub rendszerre.

VALÓSZÍNŰ Queue és pub/sub közötti döntés?

Queue esetén egy message tipikusan egy consumer group egy tagjához kerül és feldolgozási munkát oszt el. Pub/sub egy eventet több subscribernek fan-outol. Tartós queue retryt és DLQ-t adhat; egyszerű Redis pub/sub elveszítheti az üzenetet offline consumer esetén. Az üzleti követelmény: delivery, ordering, replay és fan-out dönti el.

Várható follow-upok

- Mikor SQS és mikor SNS/EventBridge?
- Mi a consumer group?
- Hogyan kezelsz poison message-et?

Buktató Ne technológianévvvel kezdj; előbb delivery semantics.

VALÓSZÍNŰ Mi az outbox pattern?

Az üzleti adat és a kiküldendő event ugyanabban az adatbázis-tranzakcióban kerül mentésre egy outbox táblába. Külön publisher olvassa és továbbítja a brokerbe, majd jelöli feldolgozotttnak. Így elkerülhető, hogy DB commit sikeres, event publish viszont sikertelen legyen. A consumer továbbra is legyen idempotens. **Várható follow-upok**

- Hogyan takarítod az outboxot?
- Hogyan kezeled a duplicate publish-t?
- Mi a CDC alternatíva?

Buktató Az outbox nem ad automatikus exactly-once end-to-end garanciát.

VALÓSZÍNŰ Hogyan találsz memória-szivárgást Node-ban?

Figyelem a heap növekedését terhelés után, heap snapshotokat hasonlítok, allocation profile-t és retained object graphot nézek. Tipikus ok: végtelen cache, event listener eltávolítás hiánya, closure által megtartott nagy object, timer, connection registry vagy request context. A restart csak tünetkezelés.

Várható follow-upok

- Mi a heap és RSS különbsége?
- Mit jelez az EventEmitter warning?
- Hogyan reprodukálod?

Buktató Ne csak a garbage collectort hibáztasd.

VALÓSZÍNŰ Milyen security alapokat vársz Node API-nál?

Input validation, authn/authz szétválasztás, least privilege, secret manager, dependency scanning, rate limit, request size limit, secure headers, SSRF és injection elleni védelem, audit log és érzékeny adatok maszkolása. A JavaScript ecosystemnél a supply-chain és transitive dependency kockázat különösen fontos. **Várható follow-upok**

- Hogyan védekezel prototype pollution ellen?
- Mit validálsz URL fetch előtt?
- JWT esetén mit ellenőrzöl?

Buktató A validation library önmagában nem teljes security stratégia.

KÓDOLÁSI FELADAT Concurrency limiter

Feladat Írj függvényt, amely legfeljebb N aszinkron taskot futtat egyszerre, és megőrzi az eredmények sorrendjét. **Mit figyelnek**

- Nincs korlátlan Promise.all
- Hibakezelési stratégia tisztázása
- Sorrend megőrzése
- Edge case: limit ≤ 0

```
async function mapLimit(
  items,
  limit,
  worker,
) {
  if (!Number.isInteger(limit) ||
    limit < 1) {
    throw new Error(
```

```

    "limit must be positive",
  );
}

const results =
  new Array(items.length);
let next = 0;

async function run() {
  while (true) {
    const index = next++;
    if (index >= items.length) {
      return;
    }
    results[index] = await worker(
      items[index],
      index,
    );
  }
}

const count = Math.min(
  limit,
  items.length,
);
const runners = Array.from(
  { length: count },
  () => run(),
);

await Promise.all(runners);
return results;
}

```

Megbeszélendő trade-offok

- Fail-fast helyett allSettled-szerű eredmény is kérhető.
- Cancellation és retry külön követelményként adható hozzá.

4. REST, HTTP, adat és security

A REST kérdések gyakran egyszerűnek indulnak, majd idempotency, versioning, error model, caching és security irányba

mélyülnek.

ESSENTIAL Mi a REST definíciója?

A REST egy architectural style elosztott hypermedia rendszerekhez. Fő constraintjei: client-server, stateless, cacheable, uniform interface, layered system és opcionális code-on-demand. Egy JSON-over-HTTP API nem automatikusan RESTful; a uniform interface és a resource-orientált szemlélet a lényeg. **Várható follow-upok**

- Mit jelent a stateless?
- Mi a uniform interface?
- Kell-e HATEOAS minden gyakorlati API-hoz?

Buktató Ne definiáld úgy, hogy „REST az, amikor HTTP metódusokat használunk”.

ESSENTIAL Mely HTTP metódusok idempotensek?

GET, PUT, DELETE, HEAD és OPTIONS szemantikailag idempotens: ugyanazt a kérést többször végrehajtva a kívánt szerverállapot ugyanaz. POST általában nem idempotens, de idempotency key-jel azzá tehető egy adott műveletre. Idempotens nem jelent mellékhatásmentest: log vagy metric többször változhat. **Várható follow-upok**

- Mi a safe method?
- Miért lehet DELETE idempotens, ha másodszor 404?
- PATCH idempotens?

Buktató Ne keverd az idempotenciát azzal, hogy minden response bitre azonos.

ESSENTIAL PUT vagy PATCH?

PUT tipikusan egy erőforrás teljes reprezentációját cseréli vagy ismert URI-n létrehozza, és idempotens. PATCH részleges módosítást fejez ki; idempotenciája a patch formátumtól és művelettől függ. A szerződésben egyértelművé kell tenni a hiányzó, null és törlendő mezők jelentését.

Várható follow-upok

- JSON Merge Patch vagy JSON Patch?
- Hogyan kezeled a concurrent update-et?

Buktató Ne használj PATCH-et homályos partial objectként explicit szemantika nélkül.

ESSENTIAL Hogyan tervezel egységes API error response-t?

Legyen stabil machine-readable code, emberi message, correlation ID és opcionális field-level details. HTTP status a protokollsintű kategóriát jelzi, domain code a konkrét okot. Stack trace, SQL és secret ne kerüljön a klienshez. A logban több context lehet, de adatvédelmi szabályokkal.

Várható follow-upok

- 400 vagy 422?
- 409 mikor indokolt?
- Hogyan támogatod a localizationt?

Buktató Ne tedd a kliens logikáját szabad szöveges message parsingra.

ESSENTIAL Hogyan verziózol API-t és mi breaking change?

Breaking change az, ami egy meglévő korrekt klienst módosítás nélkül elronthat. URL vagy header versioning használható; fontosabb a kompatibilitási policy, deprecation és mérés. Mező eltávolítása vagy jelentésének változása breaking. Új opcionális mező általában kompatibilis, de zárt enum kliensnél új enum value is törést okozhat. **Várható follow-upok**

- Breaking-e egy új enum érték?
- Mennyi ideig tartod a régi verziót?
- Hogyan mérnéd a fogyasztókat?

Buktató Ne tekintsd automatikusan non-breakingnek az összes additív változást.

ESSENTIAL Offset vagy cursor pagination?

Offset egyszerű és alkalmas stabil, kisebb datasetre, de nagy offset drága lehet, és concurrent insert/delete miatt duplicate vagy kihagyás jöhet. Cursor stabil rendezési kulcsból folytat, skálázhatóbb és konzisztensabb feedhez. A cursor legyen opaque és deterministic order kell, például createdAt plus id. **Várható follow-upok**

- Hogyan lapoznál visszafelé?
- Mit tartalmaz a cursor?
- Mi történik azonos timestampnél?

Buktató Ne használj csak timestampet tie-breaker nélkül.

VALÓSZÍNŰ Hogyan működik az optimistic concurrency?

A kliens elküldi az általa olvasott versiont vagy ETag-et, a szerver pedig csak akkor módosít, ha az aktuális verzió egyezik. Eltérésnél 409 vagy 412 jelzi a konfliktust. Így elkerülhető a lost update anélkül, hogy hosszú lockot tartanánk. **Várható follow-upok**

- If-Match és ETag?
- Mikor pessimistic lock?
- Hogyan merge-elni?

Buktató Ne írj felül csendben egy másik user frissítését.

ESSENTIAL JWT esetén mit ellenőrizöl?

Signature, engedélyezett algorithm, issuer, audience, expiry, not-before és a szükséges claim-ek. Az authentication csak identitás; az authorization minden műveletnél külön ellenőrzés. Rövid access token, secure refresh flow és key rotation kell. Ne támaszkodj csak a frontend által küldött role-ra. **Várható follow-upok**

- Hol tárolnád browserben?
- Hogyan revoke-olsz?
- Mi az OAuth és OIDC különbsége?

Buktató Ne decode-old verification nélkül, és ne fogadj el dinamikusan bármilyen algoritmet.

VALÓSZÍNŰ Hogyan tervezel rate limitet?

Kulcs lehet user, tenant, token, IP vagy endpoint kombináció. Token bucket vagy leaky bucket gyakori. Distributed rendszerben közös store vagy gateway kell, és figyelni kell a burstöt, retry-after headert, belső service limitet és abuse detectiont. Az IP önmagában NAT miatt igazságtalan lehet.

Várható follow-upok

- Mi a token bucket?
- Hol implementálnád: app vagy API gateway?
- Mi történik Redis kiesésnél?

Buktató Ne csak globális requests-per-minute számot mondj üzleti kontextus nélkül.

VALÓSZÍNŰ Hogyan cache-elnél HTTP-n?

Cache-Control határozza meg a freshness és shared/private viselkedést. ETag vagy Last-Modified conditional requestet tesz lehetővé. Csak helyesen kategorizált, jogosultság szempontból biztonságos response cache-elhető. Invalidationnál a resource változás és tenant boundary kritikus. **Várható follow-upok**

- Mi a 304?
- public vs private cache?
- Hogyan cache-elnél user-specifikus adatot?

Buktató Ne cache-elj authorization-függő választ közös kulcs alatt.

VALÓSZÍNŰ SQL tranzakció és isolation mire jó?

Tranzakció atomikusságot ad az összetartozó módosításokhoz. Isolation szabályozza, milyen concurrent anomáliák láthatók. Read committed sok rendszerben jó alap, de lost update vagy write skew ellen explicit locking, version check vagy magasabb isolation kellhet. A tranzakció legyen rövid és ne tartalmazzon lassú külső API-hívást.

Várható follow-upok

- Mi a dirty/non-repeatable/phantom read?
- Mikor használod SELECT FOR UPDATE-et?
- Miért baj a külső API a tranzakcióban?

Buktató Ne mondd, hogy ACID minden distributed folyamatot automatikusan megold.

KÓDOLÁSI FELADAT Idempotens shipment létrehozás

Feladat Tervezd meg a POST /shipments endpointot úgy, hogy kliens retry esetén ne jöjjön létre duplicate shipment. **Mit figyelnek**

- Idempotency-Key header
- Kulcs + tenant unique constraint
- Request fingerprint ellenőrzés
- Stored response vagy resource ID
- Concurrent request kezelése

Megbeszélendő trade-offok

- A key lejáratí ideje üzleti döntés.
- Ugyanaz a key más request bodyval

konfliktus.

- A DB unique constraint a race condition utolsó védelmi vonala.

5. AWS és cloud architecture

Nem AWS szolgáltatáslistát kell mondani, hanem követelményből dönteni: traffic pattern, latency, state, deployment, operációs teher, security és költség.

ESSENTIAL Lambda vagy ECS/Fargate?

Lambda jó eseményvezérelt, változó vagy rövid workloadra, gyors scale-outtal és alacsony operációs teherrel. Limitje lehet startup latency, execution limit, connection kezelés és runtime kontroll. ECS/Fargate jobb hosszú futású service, stabil traffic, WebSocket, custom process vagy nagyobb kontroll esetén. A döntést workload és csapatüzemeltetés alapján hozom. **Várható follow-upok**

- Mikor zavar a cold start?
- Hogyan kezelsz DB connectiont Lambdából?
- Miért nem ideális Lambda tartós WebSocket gatewaynek?

Buktató Ne válassz Lambdát csak azért, mert serverless, vagy ECS-t csak azért, mert ismerős.

ESSENTIAL API Gateway vagy ALB?

API Gateway managed API feature-öket ad: auth integration, throttling, usage plan, request transformation és serverless integráció. ALB költséghatékony lehet stabil nagy forgalomnál és container service-eknél, és natívan load balance-ol. A protokoll, feature igény, traffic és pricing dönt. **Várható follow-upok**

- WebSocket API?
- Private API?
- Hol végzed a rate limitet?

Buktató Ne kezeld őket teljesen felcserélhetőnek.

ESSENTIAL RDS/Aurora vagy DynamoDB?

RDS/Aurora jó relációkhoz, ad-hoc queryhez, transactionhoz és SQL ökoszisztémához. DynamoDB akkor erős, ha access pattern előre ismert, key-value/document modell elég, és nagyon rugalmas scale kell. DynamoDB-nél a partition key, hot partition és GSI tervezés központi. Nem „SQL rossz, NoSQL gyors” döntés. **Várható follow-upok**

- Mikor kell transaction DynamoDB-ben?
- Mi az N+1 SQL probléma?
- Hogyan választasz partition keyt?

Buktató Ne válassz adatbázist kizárólag névleges skálázhatóság alapján.

ESSENTIAL SQS, SNS vagy EventBridge?

SQS tartós queue és work distribution, retryval és DLQ-val. SNS push-alapú fan-out több subscriberhez. EventBridge eseménybusz routinggal, filteringgel és sok AWS/SaaS integrációval. Gyakori kombináció: publisher -> SNS/EventBridge -> subscriberenként SQS, így fan-out és consumer isolation is van. **Várható follow-upok**

- Standard vagy FIFO SQS?
- Mi a visibility timeout?
- Mikor kell DLQ redrive?

Buktató Ne állíts pontosan egyszeri deliveryt Standard SQS-re; a consumer legyen idempotens.

ESSENTIAL Hogyan tervezel VPC-t egy webalkalmazáshoz?

Public subnetben csak internet-facing load balancer vagy NAT gateway jellegű komponens legyen. App és DB private subnetben több AZ-ban. Security group role-alapú, szűk kapcsolatokat ad. Private AWS service eléréshez VPC endpoint csökkentheti a NAT igényt. Route table és DNS megértése fontos. **Várható follow-upok**

- Mi a NAT gateway szerepe?
- Security group vs NACL?
- Miért kell több AZ?

Buktató A private subnet nem azt jelenti, hogy nincs outbound internet

automatikusan.

ESSENTIAL Mit jelent IAM least privilege?

Minden workload és ember csak a szükséges actiont kapja a szükséges resource-ra, megfelelő conditionnel. Hosszú életű access key helyett role és temporary credential. A permission boundary, SCP és resource policy külön rétegek lehetnek. A wildcardot csak indokolt, kontrollált esetben használom.

Várható follow-upok

- Identity-based vs resource policy?
- Hogyan adsz cross-account hozzáférést?
- Mi az assume role?

Buktató Ne adj AdministratorAccess-t deployment egyszerűsítésére productionben.

VALÓSZÍNŰ Hogyan építész resilient service-t AWS-ben?

Multi-AZ deployment, stateless instances, health checks, autoscaling és megfelelő timeoutok az alap. Dependency hívásoknál retry csak idempotens műveletre, exponential backoff+jitter, circuit breaker és bulkhead. Queue kisimítja a burstöt. A degradált működést és recovery procedure-t előre tervezem. **Várható follow-upok**

- Mi a különbség reliability és resilience között?
- Mikor árt a retry storm?
- Mit jelent a bulkhead?

Buktató Ne azonosítsd a resilience-t azzal, hogy „van két instance”.

ESSENTIAL Milyen observabilityt építesz?

Structured log correlation ID-val, üzleti és technikai metric, distributed trace és alert. Golden signals: latency, traffic, errors, saturation. Dashboard release és tenant szerint szeletelhető. Alert ne pusztán CPU-ra, hanem user impactre és SLO burn rate-re épüljön. **Várható follow-upok**

- Mi a log, metric és trace szerepe?
- Mi az SLI/SLO?
- Hogyan követed végig a requestet queue-n át?

Saját irányod Prometheus/Grafana, Sentry és Kibana tapasztalatot konkrét dashboarddal vagy incidenttel kösd össze.

Buktató Ne logolj mindent kontroll nélkül; költség, zaj és személyes adat is kockázat.

VALÓSZÍNŰ Blue/green vagy canary deployment?

Blue/green két teljes környezet között vált, gyors rollbackkel, de költségesebb és state/schema kompatibilitást igényel. Canary fokozatosan növeli az új verzió trafficját, metric gate-ekkel. Mindkettőnél backward-compatible DB migration és automatizált rollback jel szükséges.

Várható follow-upok

- Hogyan migrálsz adatbázist zero

downtime-mal?

- Milyen metric állítja meg a canaryt?
- Mi a feature flag szerepe?

Buktató A deployment stratégia nem oldja meg a breaking schema change-et.

VALÓSZÍNŰ Miért Infrastructure as Code?

Reprodukálható, review-zható és verziózott infrastruktúrát ad, csökkenti a kézi driftet és támogatja a több environmentet. Modulokat csak valódi ismétlődésre hozok létre. Secret ne kerüljön state-be vagy repositoryba. A deployment pipeline validál, plan/diffet mutat és kontrolláltan alkalmaz. **Várható follow-upok**

- CloudFormation/SAM vagy Terraform?
- Hogyan kezeled a driftet?
- Mi legyen environment-specifikus?

Buktató Ne legyen a production infrastruktúra konzolból kézzel módosított „snowflake”.

KÓDOLÁSI FELADAT AWS architektúra: shipment status pipeline

Feladat Tervezd meg: partner API-k status eventeket küldenek, normalizálni kell, eltárolni, UI-n közel valós időben megjeleníteni, hibás eventet újrapróbálni.

Mit figyelnek

- Ingress auth és rate limit
- Queue a burst és retry kezelésére

- Idempotens consumer
- DLQ és replay
- DB access pattern
- WebSocket/SSE update
- Metrics és tracing

Megbeszélendő trade-offok

- Lehetséges: API Gateway -> Lambda/ECS ingest -> SQS -> consumer -> DynamoDB/RDS -> EventBridge/SNS -> WebSocket gateway.
- A konkrét szolgáltatásnál fontosabb a delivery és failure semantics.

6. CI/CD, tesztelés és production hibakeresés

A hirdetés Git és CI/CD tapasztalatot is kér. Technikai interjún azt nézhetik, hogyan jut el biztonságosan a kód productionbe, és hogyan találd meg gyorsan a hibát.

ESSENTIAL Milyen pipeline-t építenél?

Gyors lint/typecheck/unit test az elején, majd build, dependency/security scan, integration/contract teszt, artifact publish és environment deployment. Ugyanaz az immutable artifact halad előre. Production előtt approval vagy automatizált quality gate lehet; utána smoke test, metric monitoring és rollback. **Várható follow-upok**

- Mit futtatsz PR-en és mit merge után?
- Hogyan gyorsítod a pipeline-t?
- Mi az artifact promotion?

Buktató Ne rebuildeld eltérően ugyanazt a verziót minden environmentben.

VALÓSZÍNŰ Trunk-based vagy hosszú feature branchek?

Trunk-based kis, gyakori merge-eket és feature flaget használ, csökkenti az integrációs kockázatot. Hosszú branch nagy merge konfliktust és késői feedbacket okoz. Szabályozott környezetben release branch lehet indokolt, de a default rövid branch és automatizált teszt. **Várható follow-upok**

- Hogyan kezeled a félkész funkciót?
- Mikor törlöd a feature flaget?

Buktató A feature flag ne váljon tartós, dokumentálatlan konfigurációs adóssággá.

ESSENTIAL Mi a unit, integration, contract és E2E teszt szerepe?

Unit gyorsan ellenőrzi a tiszta logikát. Integration valódi adapterek vagy kontrollált infrastruktúra együttműködését. Contract a provider-consumer kompatibilitást. E2E kevés kritikus üzleti flow-t fed le. A cél nem százalék, hanem megfelelő kockázat megfelelő szinten. **Várható follow-upok**

- Mit mockolnál?
- Hogyan tesztelsz queue consumert?
- Mikor kell testcontainer?

Buktató Ne legyen minden teszt E2E, de ne mockolj ki minden integrációt sem.

ESSENTIAL Hogyan debugolsz production incidentet?

Először scope és user impact, majd idővonal: mikor kezdődött, történt-e release vagy dependency változás. Dashboardon latency/error/saturation, logban correlation ID, trace-ben bottleneck. Rövid távú mitigation, például rollback vagy feature disable; utána root cause és preventív action. **Várható follow-upok**

- Mit csinálsz, ha nincs reprodukció?
- Hogyan különíted el app és dependency hibát?
- Mi kerül postmortembe?

Saját irányod Hozz Sentry/Kibana vagy Prometheus/Grafana konkrét esetet.

Buktató Ne kezdj vak logolás- vagy kódmódosításba hipotézis nélkül.

VALÓSZÍNŰ Mit jelent az SLO és error budget?

SLI a mért indikátor, például sikeres request arány vagy latency. SLO a cél, például 99.9 százalék. Error budget a megengedett hibakeret, amely segít egyensúlyozni feature delivery és reliability munkát. A business-critical flow-khoz külön SLO érdemes. **Várható follow-upok**

- Miért nem 100 százalék?
- Mi a burn-rate alert?

- Milyen SLI lenne shipment trackinghez?

Buktató Ne keverd az SLA jogi vállalását a belső SLO-val.

VALÓSZÍNŰ Hogyan vezetsz be breaking adatbázis-változást?

Expand-contract: először additív schema, amelyet régi és új kód is kezel. Ezután backfill vagy dual-write/read migration, majd új kód teljes rollout és csak végül a régi mező eltávolítása. Minden fázis visszagörgethető és mérhető. **Várható follow-upok**

- Mi történik rollbacknél?
- Hogyan kezelsz nagy backfillt?
- Mikor kell online index build?

Buktató Ne deployolj kódot, amely azonnal igényel egy még nem mindenhol elérhető schema változást.

7. LLM, MCP és AI agents - gyakorlati mélység

A szerep külön kiemeli az LLM, MCP, agents és orchestration témákat. Itt nem kell kutatónak lenned; azt kell mutatni, hogy biztonságos, mérhető és üzletileg hasznos workflow-t tudsz építeni.

ESSENTIAL Mi a különbség AI workflow és agent között?

Workflowban a lépések és elágazások nagyrészt előre definiáltak. Agentnél a modell dinamikusan választ következő lépést vagy toolt a cél és állapot alapján. Determinisztikus üzleti folyamatnál workflow az első választás; agent csak ott, ahol valódi bizonytalanság és adaptív döntés ad értéket.

Várható follow-upok

- Mikor túlzás agentet használni?
- Hogyan korlátozod a lépésszámot?
- Mikor kell human approval?

Buktató Ne nevezd agentnek az egyszerű egyetlen prompt + API call folyamatot.

ESSENTIAL Hogyan épül fel egy tool-calling loop?

A host elküldi a user célt, system policyt és elérhető tool schema-kat. A modell tool callt javasol strukturált argumentummal. Az alkalmazás validálja az argumentumot, ellenőrzi az authorizationt és esetleg user approvalt kér, majd végrehajtja. Az eredmény visszakerül a modellhez, amíg kész válasz vagy stop condition nem jön.

Várható follow-upok

- Hol történik a validation?
- Mi a maximum iteration?
- Hogyan kezelsz tool timeoutot?

Buktató A modell tool javaslata nem authorization döntés.

ESSENTIAL Mi az MCP magas szintű architektúrája?

Az MCP host az AI alkalmazás, amely több MCP clientet kezel; minden client egy szerverrel tart kapcsolatot. A server tools, resources és prompts primitíveket kínál. A protokoll JSON-RPC alapú, lifecycle és capability negotiation van. Helyi kapcsolatnál STDIO, távoli szervernél Streamable HTTP használható. **Várható follow-upok**

- Mi a host és client különbsége?
- Tool vs resource vs prompt?
- Miért stateful a session?
- Mi történik initializationkor?

Buktató Ne mondd, hogy az MCP maga agent framework vagy modell API; context és capability exchange protokoll.

ESSENTIAL Tool, resource és prompt MCP-ben?

Tool végrehajtható művelet, amelyet a modell vagy host kezdeményezhet. Resource passzív context adat, például schema vagy dokumentum. Prompt újrafelhasználható interakciós template, tipikusan user által kiválasztva. A biztonsági és UX modelljük eltér, ezért nem érdemes mindent toolként kitenni. **Várható follow-upok**

- Miért jó read-only adatnak resource?
- Hogyan discovereli a client a toolokat?
- Mit tartalmaz a tool input schema?

Buktató Ne adj write access-t resource-nak vagy kontrollálatlan side effectet egy homályos toolnak.

VALÓSZÍNŰ Milyen security kockázatai vannak MCP/agent rendszernek?

Prompt injection, túl széles tool permission, data exfiltration, confused deputy, SSRF, malicious tool output és supply-chain kockázat. Védelem: least privilege, allowlist, schema validation, tenant isolation, output sanitization, approval veszélyes művelet előtt, audit trail, secret boundary és untrusted content egyértelmű kezelése.

Várható follow-upok

- Mi az indirect prompt injection?
- Hogyan választod szét a user és tool jogosultságot?
- Mit logolsz?

Buktató A system prompt nem security boundary.

ESSENTIAL Mi a RAG és mikor használnád?

Retrieval-augmented generation releváns külső dokumentumot keres és contextként ad a modellnek. Akkor jó, ha a tudás változik, forrásokhoz kötött válasz kell, vagy a teljes corpus nem fér contextbe. A retrieval minőség, chunking, permission filtering, citation és evaluation kritikus.

Várható follow-upok

- Embedding vagy keyword search?

- Hogyan kerülöd a cross-tenant leakage-et?
- Mi a reranking?

Buktató A RAG nem garantál helyes választ; rossz retrieval rossz contextet ad.

ESSENTIAL Hogyan tesztelsz LLM feature-t?

Rögzített eval dataset reprezentatív és adversarial példákkal, strukturált metric: task success, groundedness, tool correctness, latency és cost. Offline eval mellett shadow/canary és human review kell. Prompt, model és tool verziót rögzíteni kell, mert a viselkedés ezek függvénye. **Várható follow-upok**

- Mi legyen golden dataset?
- Hogyan tesztelsz nondeterminismust?
- Mikor elég exact match?

Buktató Ne csak néhány kézi prompt alapján mondj production-ready minőséget.

VALÓSZÍNŰ Hogyan kezeled a costot és latencyt?

Kisebb modell egyszerű lépésekre, routing a komplexitás szerint, context trimming, cache, batch, streaming és párhuzamos független tool hívások. Limit tokenre, iterationre és tool timeoutra. Metric tenant és use case szerint; business value nélkül a gyorsabb vagy olcsóbb modell sem siker. **Várható follow-upok**

- Mikor cache-elhető egy LLM response?

- Mi a first-token latency?
- Hogyan degradálsz model outage esetén?

Buktató Ne küldj automatikusan teljes adatbázisrekordot vagy hosszú chat historyt minden kéréshez.

VALÓSZÍNŰ Milyen AI use case illik logisztikához?

Operációs exception triage: összegyűjti a shipment státuszt, partner üzeneteket és SLA-kat, majd javaslatot készít az operátornak. Toolok olvasnak trackinget és tarifát; write művelet, például ügyfélértesítés vagy átütemezés approvalhoz kötött. A siker metricje resolution time és accepted recommendation, nem a „szép válasz”.

Várható follow-upok

- Mit automatizálnál teljesen?
- Mi legyen human-in-the-loop?
- Hogyan auditálsz a döntést?

Buktató Ne kezd autonóm pénzügyi vagy ügyfélhatású művelettel kontroll nélkül.

KÓDOLÁSI FELADAT MCP tool tervezés

Feladat Tervezd meg a `get_shipment_status` és `reschedule_shipment` toolokat. Milyen input schema, permission és output kell?

Mit figyelnek

- Szűk, egyértelmű tool neve
- Tenant/user authorization minden callnál
- Read és write külön tool

- Write előtt approval vagy dry-run
- Structured output és domain error
- Idempotency a reschedule-nél

Megbeszélendő trade-offok

- A tool description segíti a modellt, de nem security kontroll.
- A shipment ID-t szerveroldalon a caller tenantjához kell kötni.

8. System design - hogyan vezesd a beszélgetést

Az interjúztató nem egyetlen „helyes AWS diagramot” keres. A módszer, a feltételezések, a trade-offok, a failure mode-ok és az evolúciós út számít.

System design válaszváz

1. Tisztázd a use case-t és a scope-ot.
2. Mondj becsült terhelést és fontos SLA/SLO-t.
3. Nevezd meg a fő adatmodelleket és API-kat.
4. Rajzold fel a high-level komponenst és data flow-t.
5. Mélyíts a legkockázatosabb részen.
6. Beszélj failure-ről, idempotenciáról, securityről és observabilityról.

7. Zárd trade-offokkal és evolúciós lépésekkel.

ESSENTIAL Tervezd meg a shipment tracking platformot.

Követelmény: partner eventek fogadása, normalizálás, aktuális állapot és history, kereshető UI, közel valós idejű update. Ingress auth + rate limit, durable queue, idempotens normalizer/consumer, event store vagy history table, current-state projection, REST search és WebSocket/SSE update. DLQ, replay, schema version, correlation ID és tenant isolation szükséges.

Várható follow-upok

- Hogyan kezelsz out-of-order eventet?
- Mi a shipment státusz source of truth-ja?
- Hogyan oldod meg a search-et?
- Mi történik partner outage esetén?

Buktató Ne ugorj egyből AWS szolgáltatásnevekre követelmények nélkül.

ESSENTIAL Hogyan kezelnéd az out-of-order és duplicate status eventeket?

Minden eventnek legyen partner event ID-ja, shipment ID-ja, event time-ja és ingest time-ja. Deduplikáció unique key vagy idempotency store alapján. Orderinget shipment partitionon belül lehet őrizni, de későn érkező eventre domain szabály kell: verzió, sequence number vagy állapotgép. Historyt megőrizzük, current projectiont

determinisztikusan frissítjük. **Várható follow-upok**

- Mi történik, ha nincs sequence number?
- Event time vagy ingest time?
- Hogyan javítod a hibás projectiont?

Buktató Ne feltételezd, hogy a hálózat és broker globális sorrendet garantál.

VALÓSZÍNŰ Tervezd meg a pricing recommendation szolgáltatást.

A synchronous quote API gyors baseline árat ad cache-elt rule/model alapján. A drágább feature computation és model update aszinkron pipeline. Minden ajánlathoz model/rule version és input snapshot kerül audit célra. Fallback egyszerű szabályalapú árra, ha AI/model dependency hibás. A/B vagy canary és business metric szükséges.

Várható follow-upok

- Mennyi latency fér bele?
- Hogyan kerülöd a túl régi adatot?
- Hogyan magyarázod az árat?
- Mi a fallback?

Buktató Ne tedd az LLM-et kritikus árképzési hot pathba determinisztikus guardrail nélkül.

VALÓSZÍNŰ Tervezd meg az AI operations assistantet.

A host autentikált user sessionnel működik, és tenant-szűrt read-only MCP toolokat kap.

Retrieval hozza a releváns shipment és policy adatot. A modell javaslatot készít; write tool külön scope és user approval után fut. Minden prompt/tool/model verzió és tool call auditált. Step, time és cost limit van, valamint fallback hagyományos UI-ra.

Várható follow-upok

- Hogyan védekezel prompt injection ellen?
- Hogyan értékeled a hasznosságot?
- Mi történik modell outage-nél?

Buktató Ne add a modellnek közvetlen, korlátlan adatbázis- vagy admin hozzáférést.

Gyors kapacitásbecslés

Példa: 2 millió shipment/nap, shipmentenként átlag 12 event -> 24 millió event/nap. Ez átlagosan kb. 278 event/s, de partner batch és csúcs miatt tervezz például 10-20-szoros bursttel. Az átlag önmagában nem méretezési alap. Mondd ki a payload méretet, retentiont, read/write arányt és hot tenant kockázatot.

DEEP DIVE Mikor választanál event-driven architektúrát?

Ha a producer és consumer életciklusát szét kell választani, burstöt kell puffereelni, több downstream reakció van vagy replay/audit fontos. Az ára eventual consistency, nehezebb debugging, schema evolution és duplicate handling. Egyszerű synchronous CRUD flow-t nem bontanék eventekre csak

divatból. **Várható follow-upok**

- Mikor request-response jobb?
- Hogyan követed a distributed transactiont?
- Mi a saga?

Buktató Ne használj eventet olyan helyen, ahol azonnali, erős konzisztens válasz az üzleti igény.

9. Saját technikai történetbank

A jó technikai válasz hiteles projektpéldával zárul. Az alábbi vázak a saját tapasztalatodra épülnek; csak olyan számot mondj, amelyet vállalsz és meg tudsz magyarázni.

ESSENTIAL Skálázható WebSocket support chat - történetváz

Helyzet: legacy rendszer és protokoll, skálázhatósági és fenntarthatósági korlátok. Feladat: modern Node.js szolgáltatás, amely több instance-on is konzisztensen kezeli a connectionöket. Döntések: protokoll reverse engineering, JWT, Redis pub/sub, stateless gateway, reconnect és rollout stratégia. Eredmény: kisebb latency és alacsonyabb infrastruktúra-költség, stabilabb skálázás.

Várható follow-upok

- Miért Redis pub/sub?
- Milyen delivery garancia volt?

- Hogyan kezelted a slow clientet?
- Mi volt a legnagyobb hiba a migráció során?

Buktató Tisztázd, mely részeket tervezted vagy implementáltad te személyesen.

ESSENTIAL Nagy React/TypeScript SPA - történetváz

Helyzet: üzletileg kritikus, hosszú életű frontend sok stakeholderrel. Feladat: end-to-end feature delivery, reusable komponensek, API integráció, release koordináció és production monitoring. Döntések: TypeScript szerződések, state és effect határok, Storybook, feature flags, Sentry/Kibana, fokozatos rollout. Eredmény: gyorsabb és biztonságosabb release, jobb diagnosztika. **Várható follow-upok**

- Hogyan választottál state managementet?
- Mi volt egy performance bottleneck?
- Hogyan kezeltél breaking API change-et?

Buktató Ne maradj általános; válassz egy konkrét feature-t vagy incidentet.

VALÓSZÍNŰ Node automation és Confluence - történetváz

Helyzet: ismétlődő riport- és dokumentációs munka, valamint build asset anomáliák. Feladat: automatizált adatgyűjtés, validáció és Confluence REST publikálás. Döntések: idempotens futás, authentication, retry, részleges hiba, strukturált output és audit.

Eredmény: kevesebb manuális munka és kisebb production package. **Várható follow-upok**

- Hogyan kezelted az API rate limitet?
- Mi történt félbeszakadt futásnál?
- Hogyan validáltad az eredményt?

Buktató Ne csak azt mondd, hogy scriptet írtál; mutasd a megbízhatósági döntéseket.

VALÓSZÍNŰ Event broadcaster modernizáció - történetváz

Helyzet: régi Node runtime és nagy concurrent connection szám. Feladat: modernizáció downtime nélkül. Döntések: Node upgrade, JWT, PM2 cluster, rolling update, Prometheus/Grafana és load test. Eredmény: nagyobb throughput, alacsonyabb p95 latency és jobb operálhatóság. **Várható follow-upok**

- Hogyan mértél előtte és utána?
- Mi volt CPU és memory profil?
- Hogyan rollbackeltél volna?

Buktató A százalékokat csak akkor mondd, ha a mérési módszert is el tudod magyarázni.

VALÓSZÍNŰ GraphQL integráció - történetváz

Helyzet: frontendnek túl nagy vagy nem optimális payloadok és több integrációs igény. Feladat: GraphQL használata a szükséges mezők lekéréséhez és a kliens

flow integrálásához. Döntések: query fragmentek, cache policy, error/partial data, type generation és monitoring. Eredmény: kisebb payload és célzottabb adatlekérés.

Várható follow-upok

- Mi az N+1 a szerveren?
- Mikor REST jobb?
- Hogyan kezelted a cache invalidationt?

Buktató Ne állítsd, hogy GraphQL mindig gyorsabb vagy kevesebb requestet jelent.

10. Live coding és code review gyakorlás

Live codingnál mondd ki a feltételezéseket, kezdj egyszerű korrekt megoldással, majd beszélj edge case-ről, complexityről, tesztről és production változatról.

KÓDOLÁSI FELADAT Typed groupBy

Feladat Írj generic groupBy függvényt, amely egy selector alapján Map-be csoportosít. **Mit figyelnek**

- T és K generic kapcsolat
- K használható Map kulcsként
- Input nem mutálódik
- O(n) idő

```
function groupBy<T, K>(  
  items: readonly T[],  
  keyOf: (item: T) => K,  
) : Map<K, T[]> {  
  const result =
```

```

    new Map<K, T[]>();

    for (const item of items) {
        const key = keyOf(item);
        const group = result.get(key);

        if (group) {
            group.push(item);
        } else {
            result.set(key, [item]);
        }
    }

    return result;
}

```

Megbeszélendő trade-offok

- Object helyett Map tetszőleges kulcsot és tisztább prototípus-viselkedést ad.

KÓDOLÁSI FELADAT Retry backoffal

Feladat Írj retry wrapper-t maximum próbálkozással, exponential backoffal és retry predicate-tel. **Mit figyelnek**

- Csak retryolható hiba
- Max attempts tiszta szemantika
- Jitter említése
- Cancellation lehetőség

```

async function retry(
    fn,
    {
        attempts,
        baseMs,
        shouldRetry,
    },
) {
    let lastError;

    for (
        let attempt = 1;
        attempt <= attempts;
        attempt++
    ) {
        try {
            return await fn(attempt);

```

```

    } catch (error) {
      lastError = error;
      const finalAttempt =
        attempt === attempts;

      if (finalAttempt ||
        !shouldRetry(error)) {
        throw error;
      }

      const delay =
        baseMs * 2 ** (attempt - 1);
      const jitter =
        Math.random() * delay * 0.2;

      await new Promise(resolve =>
        setTimeout(
          resolve,
          delay + jitter,
        ),
      );
    }

    throw lastError;
  }
}

```

Megbeszélendő trade-offok

- POST csak idempotency védelemmel retryolható biztonságosan.
- Retry-After header felülírhatja a lokális backoffot.

KÓDOLÁSI FELADAT React code review

Feladat Mit javítanál: effect fetch közben nincs cleanup, array index key, derived filtered list state-ben, minden renderben új context value, error nincs kezelve? **Mit figyelnek**

- Race cancellation
- Stable key
- Derived value renderben vagy memo-val
- Context value és state boundary

- Explicit error/loading/empty state

Megbeszélendő trade-offok

- A javításokat impact szerint priorizáld, ne stílusvitával kezd.

KÓDOLÁSI FELADAT Node API code review

Feladat Mit javítanál: req.body közvetlenül SQL-be kerül, Promise nincs awaitelve, minden hiba 200 response, console.log secretet ír, nincs timeout? **Mit figyelnek**

- Schema validation és parameterized query
- Promise lifecycle és error propagation
- Helyes status/error model
- Sensitive data masking
- Outbound timeout/cancellation
- Observability

Megbeszélendő trade-offok

- Mondd el, melyik security vagy correctness blocker, és melyik maintainability javítás.

Komplexitási gyorslista

Array scan: $O(n)$. Hash map lookup átlag $O(1)$. Sort: $O(n \log n)$. Nested loop nem mindig $O(n^2)$, ha két pointer együtt halad. Network/DB feladatnál a Big-O mellett latency, round trip, payload, index és concurrency limit is számít.

11. 60 perces próbainterjú

Válaszolj hangosan. A follow-upoknál ne olvasd el előre a mintát; próbálj döntést és trade-offot megfogalmazni.

1. **0-5 perc:** Technikai bemutatkozás és a legrelevánsabb projekt.
2. **5-12 perc:** Mi a különbség any, unknown és never között? Mutass discriminated uniont.
3. **12-20 perc:** useEffect dependency, race condition és stale closure.
4. **20-28 perc:** Node event loop, CPU-intensive task és graceful shutdown.
5. **28-35 perc:** REST versioning, idempotency és pagination.
6. **35-43 perc:** Lambda vs ECS, SQS/SNS/EventBridge, observability.
7. **43-55 perc:** Shipment tracking system design, duplicate és out-of-order eventekkel.
8. **55-60 perc:** MCP architecture, agent security és egy logisztikai AI use case.

Interjúztató follow-up csomag

- Miért ezt választottad a másik megoldás helyett?
- Mi történik dependency vagy instance kiesésnél?
- Hogyan méred, hogy működik?
- Hogyan teszteled?
- Mi a security kockázat?

- Hogyan migrálnád a jelenlegi rendszert downtime nélkül?
- Mi lenne az egyszerű MVP, és mi jönne csak később?
- Mit csinálnál másképp, ha a forgalom százszorosára nő?

12. Utolsó napi technikai cheat sheet

Az utolsó napon ne új témát tanulj. Mondd el hangosan a következő mondatokat és a hozzájuk tartozó saját példát.

TypeScript: compile-time szerződés, runtime inputot validálni kell. Unknown + narrowing, discriminated union + exhaustive never.

React: render tiszta; effect külső rendszer szinkronizációja; cleanup és race cancellation. State legyen minimális és lokális.

Node: a fő szálát ne blokkoljuk. CPU -> worker/pool vagy külön service. Graceful shutdown és concurrency limit.

REST: resource, HTTP semantics, idempotency, stable error code, cursor pagination, compatibility policy.

AWS: workload alapján Lambda/ECS; durable queue; least privilege; Multi-AZ; logs+metrics+traces; safe deployment.

AI/MCP: workflow előbb, agent csak indokoltan. Model tool call nem authorization. MCP host-client-server,

tools/resources/prompts, capability negotiation.

System design: requirements -> scale -> API/data -> architecture -> failure/security/observability -> trade-offs.

Mindig: mondd ki a feltételezést, alternatívát és azt, hogyan mérnéd.

Öt saját példa, amelyet biztosan gyakorolj

- WebSocket support chat: horizontal scaling, Redis pub/sub, JWT, legacy migration.
- Nagy React/TypeScript SPA: state, reusable UI, release ownership, monitoring.
- Event broadcaster: Node upgrade, clustering, rolling deployment, Prometheus/Grafana.
- Automation: REST integration, idempotency, retry, adatvalidáció, üzleti eredmény.
- Production incident: hogyan találtad meg metric/log/trace segítségével és mit javítottál tartósan.

Kérdések, amelyeket te tehetsz fel

- How is the technical interview structured, and is there a coding or system-design exercise?
- What are the main architectural challenges in the current logistics applications?
- How is responsibility split between frontend, backend and cloud infrastructure?
- Which AWS services are central to the platform today?

- What does the team currently mean by agent-based architecture and MCP in practical product terms?
- How do you test and evaluate AI-driven workflows before they reach production?
- What would be the first technically meaningful problem for the person joining this role?

13. Források és hatókör

A kérdések a mellékelt Redspher Full Stack Developer hirdetésből, a hasonló senior full-stack technikai interjúk tipikus mélységéből és hivatalos technológiai dokumentációkból lettek összeállítva. Nem állítják, hogy ezek a Redspher tényleges, előre ismert interjúkérdései.

- Redspher Full Stack Developer job description: TypeScript, React, Node.js, AWS, REST, Git/CI/CD, LLM, MCP, agents és automation.
- React official documentation: rendering, keys, state és reactive effects.
- TypeScript Handbook: narrowing, generics és utility types.
- Node.js official documentation: event loop, worker pool és worker threads.
- AWS official architecture and deployment guidance.
- Model Context Protocol official documentation: host/client/server architecture, tools/resources/prompts, lifecycle, transports és authorization.

Készült kifejezetten a technikai interjúkörre.